

Wren Vandervelde CS263 Project

Swift and Mobile-focused runtimes (working title)

Vision Statement

My goal for this project is to investigate the Swift language and runtime, specifically in respect to the language's use in mobile development. My first goal is to re-learn the details of Swift. I've worked with this language before, but I want to re-learn it with an eye towards what we've been learning in class, such as polymorphism and garbage collection. My next goal is to learn about the levels of bytecode that swift compiles into, such as Swift Intermediate Language (SIL) and the LLVM IL. I want to see what, if any, optimizations are made to aid in mobile execution. Finally, I want to investigate swift's cross-platform nature by running a number of benchmarks on different hardware, most likely my macbook, my iphone, and a larger machine such as CSIL.

Remark

These notes were made with the typst typesetting language and the ergo package.

Source: Swift Compiler | Swift.org

<https://www.swift.org/documentation/swift-compiler/>

Source: The LLVM Compiler Infrastructure

<https://llvm.org/>

Compilation pipeline:

- Parsing - Parse code into an AST with no type checking
- Semantic analysis - Type inference and type checking; augments the AST with type information
- Clang importer - import clang modules (*note: what are these?*) and attach C / Obj-C interfaces.
- SIL generation - convert the AST into SIL (Swift Intermediate Language). *note: look at the docs later*
- SIL guaranteed transformations - something something dataflow, something something correctness? look at this later
- SIL optimizations - optimize the SIL based on swift-specific patterns.
- LLVM IR generation - generate LLVM IR
- LLVM optimization - use LLVM Core's optimizer on the IR
- Machine Code Generation - use LLVM's code generation

Based on this pipeline, now I'm a bit confused because this looks an awful lot like a compiled language, which presumably wouldn't have a runtime? There are still IRs so the project still works, but I want to get this straight. It looks like swift is compiled, but there are still references to "the Swift Runtime" across the website.

Source: About Swift | Swift.org

<https://www.swift.org/about/>

"Memory is managed automatically" "Advanced control flow with do, guard, defer, and repeat keywords"

Quote: About Swift > Platform Support

Our goal is to provide source compatibility for Swift across all platforms, even though the actual implementation mechanisms may differ from one platform to the next. The primary example is that the Apple platforms include the Objective-C runtime, which is required to access Apple platform frameworks such as UIKit and AppKit. On other platforms, such as Linux, no Objective-C runtime is present, because it isn't necessary.

Source: The Swift Runtime - Your Silent Partner

<https://blog.jacobstechtaavern.com/p/the-swift-runtime-your-silent-partner>

The above isn't a very official source, but it does contain some information on what the swift runtime actually does, which is good. "The Swift Runtime, a.k.a libswiftCore." This pretty immediately dives into SIL, so I might need to come back to it.

Source: A Swift Tour

<https://docs.swift.org/swift-book/documentation/the-swift-programming-language/guidedtour/>

I figure before I learn SIL I should brush up on swift syntax and type system.

- Pretty standard typed language. Functions are first class
- type `String?` allows for `String` values or `nil`
- closure syntax: `{ number in 3 * number }` or
`{ (number: Int) -> Int in return 3 * number }`
or even shorter `numbers.sorted { $0 > $1 }`
- inside methods, instance var access can be unqualified or use `self`.
- `init` function is named `init`. (also `deinit` exists). multiple `inits` may share function signatures, but must have different arg names.
 - Initialization has quite a few rules to ensure safety
- `class Subclass : Superclass{...}`; use `super.whatever`, manually call `super.init`. must use `override` qualifier for polymorphism.
- custom getter and setters for “fake” instance var. also `willSet` and `didSet` for code that needs to run around assignment
- enums can have methods. Use `Type.case` or just `.case` when type is inferred
- structs are passed by value, classes are passed by reference
- `async`, `await`, `Task` blocks
- protocols are kinda like rust traits, extensions implement protocols for existing types
- errors implement `Error` protocol. functions that may throw an error are marked with `throws`.
- `try?` turns errors into optionals. `defer` plays nice with `throw`.
- generics use similar syntax to Rust

Source: SIL Documentation

<https://github.com/swiftlang/swift/blob/main/docs/SIL/SIL.md>

SIL has three representations:

- In memory: SIL is represented by data structures which are implemented in the compiler sources. Optimization passes use the in-memory representation of SIL.
- Textual: The compiler and related utilities can print and parse textual SIL files. Textual SIL files have the file extension `.sil`.
- Binary: SIL can be stored and read from binary files. Binary SIL files are called “swift-module” files and have the extension `.swiftmodule`. Note that the binary format is not stable. Swift-module files are not compatible between compiler versions.

SIL Documentation Continued:

- “A .sil file is a Swift source file with added SIL definitions.”
- OSSA (Ownership Static Single-Assignment): each variable is assigned once, and ownership is checked to statically find memory leaks or use after free.
- Raw SIL vs Canonical SIL. Raw SIL may have dataflow errors and isn’t optimized yet.
- SIL types start with \$, SIL functions start with @, SIL values start with %
- SIL converts functions to basic blocks
 - all basic blocks end with a terminator
 - basic blocks take arguments
 - function arguments
 - forwarded arguments from the previous terminator
 - “phi arguments” these apparently have something to do with LLVM’s phi nodes
 - in OSSA form, all arguments have ownership annotations
- Ownership types (also starts with @, don’t confuse them with functions)
 - @owned: freestanding value is consumed exactly once during the function (by storing or destroying it typically)
 - @guaranteed: value that depends on another value’s existence, such as a borrow.
 - @unowned: “A value that is only guaranteed to be instantaneously valid.” Must be moved into @owned or @guaranteed being consumed.
 - Trivial values (such as int) don’t have ownership
- WOOO LIFETIMES
 - less complicated than rust lifetimes
 - owned lifetimes start when an owned value is passed to the block (block or function?) or when an owned value is produced by an instruction
 - guaranteed lifetimes begin at a borrow, apply, or when a guaranteed value is passed to the function.
 - a lifetime must end exactly once along every control flow path
 - the one exception is @guaranteed values passed to a function which must survive the entire function
 - interior uses are any use of a value within its lifetime which is not the start or end
 - “forward-extended lifetime”
- borrowing
 - %2 = begin_borrow %1 creates a borrow. %1 must live until %2 is unborrowed
 - load_borrow is very similar except the argument is an address and we load the memory at that address.
 - store_borrow stores an object on the stack, but has similar lifetime semantics otherwise
 - partial_apply baffles me and might make more sense when I know what apply does
- phi arguments
 - A type of argument that a basic block may take. These seem quite similar to forwarded values. I think the difference is that multiple different values may be passed to a phi argument.
 - E.g. after the end of a loop, we may be coming from either the interior of the loop or the beginning of the loop. the values from these two basic blocks need to be “merged” into one value with one lifetime.
 - speaking of lifetimes, there are some details about how owned, reborrowed and guaranteed values work
- A values must dominate instructions that use them. This essentially means that they are defined before they are used. Phi arguments make this a bit more complicated in practice.

- Joint post-dominance: sorta like lifetimes but for other things. e.g. all `alloc_stack` instructions must be followed by a `dealloc_stack` in every control path. This is called the instructions “scope”.
- linking has a bunch of semantics that I don’t quite understand
- each class has a VTable in the SIL file
- witness tables? like vtables but different. For protocols, which (as mentioned earlier) are like rust traits
 - “A witness table is emitted for every declared explicit conformance. Generic types share one generic witness table for all of their instances. Derived classes inherit the witness tables of their base class.”
 - ok I think what that means is for every pair (protocol, class) such that the class “witnesses” the protocol (follows it), there is a table mapping the various methods that the protocol expects onto the actual functions implemented in the class.
 - each protocol may have a default witness table containing requirements (methods) with “resilient” default implementations. (this has something to do with implementation order????? or is this like metaphorical dependance order)

Back on that SIL grind, I should be skimming more

- global variables exist
- stack discipline
 - all stack deallocation functions must indicate the stack allocation function they correspond to
 - also they must nest properly

Ok that was all a (informative) detour to make sure I understand SIL before continuing to learn about the runtime. Let's get back to that one article that I was looking at earlier.

Source: The Swift Runtime - Your Silent Partner

<https://blog.jacobstechtaavern.com/p/the-swift-runtime-your-silent-partner>

Since this article gives code examples, I want to see if I can follow along in `./tests/silent_partner`.

```
swiftc -emit-sil -O main.swift > sil.txt
```

Just based on what I learned from `SIL.md`, I can somewhat parse the output. We have a main function which has a single basic block, and a hidden global `@$s4main10test_classAA9TestClassCvp` which I think is just be our variable `test_class`.

This article starts by drilling down on how `alloc_ref` (which allocates space for an object (on the heap?)) is compiled.

- `IRGenSILFunction::visitAllocRefInst`
- `irgen::emitClassAllocation`
- `IRGenFunction::emitAllocObjectCall`
- `emitAllocatingCall`
- `IGM.getAllocObjectFunctionPointer()`

“When fully compiled into machine code, `[getAllocObjectFunctionPointer]` resolves as a pointer to the runtime ABI memory address of `swift_allocObject`.”

ABI is “Application Binary Interface”

The runtime is sort of like a standard library `libswiftCore`, linked to the executable “on launch” (given that swift can be compiled, is it linked at runtime? how does that work?). There is a set of standard memory locations that the swift runtime functions are at.

“On iOS, the object is instantiated with `swift_slowAllocTyped` to allocate the required heap memory for the object.” This seems like the type of stuff I’m looking for. What architectural differences does the runtime need to account for

Objects are reference counted (for garbage collection?).

Memory management notes:

- `swift/stdlib/public/runtime/HeapObject.cpp` and `Heap.cpp` are the relevant files
- `swift_allocObject` -> `swift_slowAllocTyped`/`swift_slowAlloc` -> `malloc_type_malloc/malloc`
- `swift_slowDeallocImpl` -> `AlignedFree/free`
- which is all to say that this uses the kernel’s `malloc` implementations (which on XNU (most apple products) is a red-black tree)

This has been referenced in the previous material I looked at and we actually just talked about it in class, but I want to get a good source for swift's memory management model.

Source: Automatic Reference Counting

<https://docs.swift.org/swift-book/documentation/the-swift-programming-language/automaticreferencecounting/>

Swift's ARC is a pretty standard reference counting garbage collection system. Unless otherwise specified, all references are strong references. You can, however, manually create weak references or unowned references to break strong reference cycles (which leak memory).

- Weak References: a reference that does not contribute to the RC. If the object referred to by a weak reference is deallocated, it is set to `nil` automatically.
- Unowned References: an unowned reference is guaranteed to have shorter lifetime than the object it references. It also doesn't contribute to the RC, but it never needs to be set to `nil` during runtime (since it will never be alive when the object is deallocated).
- Unsafe Unowned References: secret third option which doesn't contribute to the RC but has NO runtime checks. It has the least overhead but may point to garbage memory, which can lead to undefined behavior.

Generally, both of these forms of reference require additional work by the programmer to guarantee that an object is never used after deallocate, but weak references do that work at runtime while unowned references do that work at compile time.

Closure reference cycles!!!

Closures count as a reference to any values which they capture. This can include an object which references the closure, creating a reference cycle. To fix this you can manually specify what a closure captures and then in there annotate certain references as weak or unowned.

Example: Unowned References Test

I think that unowned references are checked statically (because otherwise what's the difference between them and unsafe unowned references) but the docs weren't super explicit about that, so I want to test it myself. In `tests/reference_counting/unowned.swift` I have a small example with a `UserAccount` which owns an `AdminAccount` and then the `AdminAccount` has an unowned reference back.

Using some functions, I want to try to force `myuser` out of scope while keeping `myadmin` around. This program compiles and then panics when we try to call `myadmin.user_account.name`.

Ok so it's not statically checked but instead is still enforced at runtime... If we change it to a weak pointer and force unwrap the variable we get a similar panic. So is `unowned` any faster? my guess is we're still going to all the `unowned` pointers and setting some flag on deallocation. Maybe it's just for programmer convenience? although it saves a single "!" character per reference, so it's not even that much more convenient. If/when I look at the implementation I can confirm these suspicions.

Last one, what does `unsafe unowned` do in the same code? No panic (as expected) and just reads garbage data (in this case an empty string).

I've left the version with `unowned` in the repo since I think it's the most interesting. Also I generated the SIL incase I ever want that.

Goal Setting

Ok, so as of writing it's Nov 20, which means I have like 1.5 weeks until the small presentation, so I want to re-evaluate what I want the primary focus of my project is. Things which I've already looked at which are interesting:

- The various IRs that swift has
- the "compiled runtime" thing that swift has going on
- swift's approach to memory management

directions I could take this:

- XNU / IOS / OSX (e.g. the fact that swift links to different mallocs per platform)
- double down on looking into mobile hardware (accelerators? signal processing? memory constraints?)
- the efficiency of the various weak pointers that swift has

I personally want to know more about the weak pointers, and also I think it'll be fun to poke around in the swift repo some more.

Unowned and Weak References Pt 2

I generated the SIL for the example I did, so first steps I want to see if there are any SIL instructions that seem relevant to either:

1. Object deallocation
2. accessing an unknown pointer

Notes:

- field defined here: `@_hasStorage unowned final let user_account: @sil_unowned UserAccount { get }` notably, there are two tags indicating unowned-ness.
- test2 expands from 3 lines to like 80
- `%1 = apply %0()` is the entire `test()` function call
- line 89 is calling printing for the first time
- printing is so large bruh
- line 103 gets the addr to the `user_account` field and then 104 loads it
- by line 106, we're accessing the fields of the `user_account` so it's all loaded by then
- relevant lines:

```
%57 = ref_element_addr [immutable] %1, #AdminAccount.user_account // user: %58
%58 = load %57 // user: %59
%59 = strong_copy_unowned_value %58 // users: %66, %60
```
- `strong_copy_unowned_value` seems like a good instruction to look at

Source: `strong_copy_unowned_value` documentation

<https://github.com/swiftlang/swift/blob/be5d4b37476799aa8aeba6f749c35af2c2539b96/docs/SIL/Instructions.md?plain=1#L1738>

Quote:

```
%l = strong_copy_unowned_value %0 : $@unowned T
```

Asserts that the strong reference count of the heap object referenced by `%0` is still positive, then increments the reference count and returns a new strong reference to `%0`. The intention is that this instruction is used as a “safe ownership conversion” from unowned to strong.

Also looking at the documentation on `load_unowned`, it seems like there are lots of reference counts per object, at minimum a strong reference count (the normal one), a weak reference count, and an unowned reference count. Worryingly, there’s no `visitStrongCopyUnownedValue` to translate the instruction into SIL, which means it’s getting transformed into something else before lowering. Likely `AddressLowering`.

After a bit more digging, the `AddressLowering` transformation is part of the SIL guaranteed transformations, which are required to occur before SIL is considered “canonical”. However, the SIL that I’ve been looking at is canonical, so `AddressLowering` should have already happened. The command `swiftc -emit-lowered-sil -O -o lowered-sil.txt unowned.swift` seems like it should give me the “most lowered” SIL, but it just errors for some reason.

Also hidden in the `SIL Instructions.md` docs is this useful line “When [the object’s] strong and unowned reference counts reach zero, the object’s memory is deallocated.”

The instruction `strong_copy_weak_value` says that it is “Lowered by `AddressLowering` to `load_weak`”. So it’s likely that `strong_copy_unowned_value` gets lowered to `load_unowned`.

Source: `load_unowned` documentation

https://github.com/swiftlang/swift/blob/be5d4b37476799aa8aeba6f749c35af2c2539b96/docs/SIL/Instructions.md#load_unowned

Quote:

```
%l = load_unowned [take] %0 : $*@sil_unowned T
```

Increments the strong reference count of the object stored at `%0`.

Decrements the unowned reference count of the object stored at `%0` if [take] is specified. Additionally, the storage is invalidated.

Requires that the strong reference count of the heap object stored at `%0` is positive. Otherwise, traps.

Looking into the LLVM implementation of `load_unowned`, `unowned_release`, and `strong_release` seem like they would be enlightening.

I'll have some more notes about how I found these details below, but I want to keep a nice table up here of how various SIL instructions get compiled, again similar to the tracing done in "Your Silent Partner".

Swift Code	<pre>class A{ let a = A()</pre>	<pre>class A{ unowned let b: B [...] } let a = A() print(a.b)</pre>	Autogenerated Deinit	Autogenerated Deinit
SIL Instruction	alloc_ref	load_unowned	unowned_release	strong_release
IR Generation Entry	<u>IRGenSIL.cpp: visitAllocRefInst</u>	<u>IRGenSIL.cpp: visitLoad##Name##Inst</u>		
IR Generation Intermediates	emitClassAllocation, emitAllocObjectCall, emitAllocatingCall	HeapTypeInfo.h: name##LoadStrong, GenHeap.cpp: emit##KIND (inside DEFINE_BINARY_OPERATION)		
libswiftCore ABI Binding	swift_allocObject	swift_unownedLoadStrong		
Runtime Notes	Depending on platform, calls malloc_type_malloc or malloc	raw dereference with a swift_unownedRetainStrong call		

Reading through swift/lib/IRGen/IRGenSIL.cpp I found out why I've been having trouble finding visitStrongCopyUnownedValue and similar functions. Turns out this file uses a bunch of macros to help define its functions, and so almost anything involving a weak/unowned pointer is hidden in SOMETIMES_LOADABLE_CHECKED_REF_STORAGE macro (or NEVER... or ALWAYS...). I'm still going to focus on load and release since I think those will be the simplest instructions which do what I want.

Ok now that I'm finally in stdlib tracing _swift_unknownObjectUnownedLoadStrong I feel close. The call stack goes (for non obj-c objects):

- swift_unknownObjectUnownedLoadStrong
- swift_unownedLoadStrong
- swift_unownedRetainStrong
- swift_nonatomic_unownedRetainStrong
- assert(object->refCounts.getUnownedCount())
- object->refCounts.tryIncrementNonAtomic()

now strong release

- swift_release
- swift_nonatomic_release
- object->refCounts.decrementAndMaybeDeinitNonAtomic(1);
- decrementAndMaybeDeinitNonAtomic
- doDecrementNonAtomic
- doDecrementNonAtomicSlow
- _swift_release_dealloc (if the reference count hits 0)
- asFullMetadata(object->metadata) ->destroy(object);

In both of the above cases, the object is a HeapObject

Week 10 Notes

2025-11-29 to 2025-12-05

Since the exact topic of my project has changed a bit over the course of the quarter, I think a more apt title is now “On the Implementation and Performance of Swift’s Weak References”

Ok time to do the actual benchmarking. I forget how much I’ve talked about this in the document but the idea is to make and benchmark tests which make a bunch of references of some type to an object.

pointer types:

- strong (basically a control)
- weak
- unowned
- unsafe unowned

tests:

- make and then drop N references to an object (create)
- read a reference to an object N times (deref)
- make reference and then drop original (destroy)

for create, $N = 1,000,000,000$ gets around 100 seconds which seems good for amortizing the load. Also I’m running the tests simultaneously too try to remove confounding variables.

Results (seconds):

test	strong	weak	unowned	unsafe unowned
create	134.962	181.800	138.643	134.682
deref	101.107	105.399	100.520	101.422
destroy	173.022	242.510	175.566	171.259
destroy - create	38.060	60.710	36.923	36.577

(NOTE TO SELF: presentation happened here)

PDF requirements

(so that I can find them easily) The PDF should include

- Link to 10-15 minute video overviewing and demo-ing project (all members must participate; multiple videos (e.g. demo separated out) are fine for turnin)
- Link to Google Sheets slides from video/demos
- Steps for building/deploying project if you did an implementation
- Link to 5 page writeup (PDF) on problem, solution, and findings
- Link to GitHub repo (shared with instructor)

post presentation thoughts:

- the benchmarks were fine for the presentation itself, but I do want to have more for the final writeup
- I can keep looking for the implementation details (I still think unowned release should have good info)
- oorr I can do some manual memory investigation try to see if I can see any difference between:
 - create object, deallocate it, create a new object
 - create object, create unowned ref to it, deallocate it, create a new object

I still want to get a straight answer from the code.

The HeapObject

in non-embedded runtimes, a `HeapObject` has two attributes, `HeapMetadata metadata` and `InlineRefCounts refCounts`. `HeapMetadata` is defined in and `InlineRefCounts` is defined in `RefCounts.h`. `RefCounts.h` has the good stuff. Let me just quote the first section. There's a bunch of other info in here.

Source: `RefCount.h`

<https://github.com/swiftlang/swift/blob/a437c6d6f6254fcd382243e202f6e60f229ba814/stdlib/public/SwiftShims/swift/shims/RefCount.h>

Quote:

An object conceptually has three refcounts. These refcounts are stored either “inline” in the field following the isa or in a “side table entry” pointed to by the field following the isa.

The strong RC counts strong references to the object. When the strong RC reaches zero the object is deinited, unowned reference reads become errors, and weak reference reads become nil. The strong RC is stored as an extra count: when the physical field is 0 the logical value is 1.

The unowned RC counts unowned references to the object. The unowned RC also has an extra +1 on behalf of the strong references; this +1 is decremented after deinit completes. When the unowned RC reaches zero the object's allocation is freed.

The weak RC counts weak references to the object. The weak RC also has an extra +1 on behalf of the unowned references; this +1 is decremented after the object's allocation is freed. When the weak RC reaches zero the object's side table entry is freed.

Objects initially start with no side table. They can gain a side table when:

- a weak reference is formed

and pending future implementation:

- strong RC or unowned RC overflows (inline RCs will be small on 32-bit)
- associated object storage is needed on an object
- etc

Gaining a side table entry is a one-way operation; an object with a side table entry never loses it. This prevents some thread races.

Strong and unowned variables point at the object. Weak variables point at the object's side table.

Side table entries hold all three reference counts, as well as a pointer to the original object.

So to summarize, there are two objects with three reference counts

- Strong and unowned references point at the main object, while weak references point to the side table
- When Strong references reach 0, `deinit()` is called, dropping any references it has, and the object is marked “deinitialized”
- After that, when unowned references reach 0, the side table's pointer is set to nil and then the `HeapObject` is freed.
- Finally, when weak references reach 0, the side table is freed.

Next Steps

This section is a loose collection of “things I want to look at later.”

definitely look at:

- memory management
 - track down the de-allocate implementation?
- benchmark different pointer types
- XNU malloc and typed malloc
- platform differences in the runtime (TARGET_OS_IOS)
- more of this one guy’s stuff: <https://blog.jacobstechtavern.com/p/what-is-a-crash?open=false#%C2%A7implementation-of-a-runtime-crash>)

less important:

- “Advanced control flow with do, guard, defer, and repeat keywords” - About Swift
- optional types
- LLVM IR
- SIL guaranteed transformations
- SIL optimizations
- LLVM optimizations